Infrastructure Documentation

Table of Contents

---

1. Project Overview

The objective is to design, deploy, and benchmark a production-representative multi-cloud Kubernetes infrastructure on-premise using an HPE ProLiant DL380 server as of now, and subsequently evaluate its performance characteristics.

The infrastructure supports a multi-tier alumni web application (React frontend, Node.js backend, MongoDB database) as the workload under evaluation.

The team progressed through three distinct infrastructure phases:

- **Phase 1 (Local, Minikube):** Initial hands-on experimentation with Kubernetes primitives on personal machines.
- **Phase 2 (Lab Desktops, Kubeadm):** First real multi-node cluster. MetalLB, Istio, Helm, and Nexus were introduced here.
- **Phase 3 (HPE ProLiant, Proxmox + Rancher):** Migration to dedicated bare-metal hardware with a production-grade Type-1 hypervisor and enterprise cluster manager.

This document covers the infrastructure management of the project from Phase 2 onwards, with a high-level note on Phase 1 for completeness.

---

2. Infrastructure Evolution Summary

Phase	Environment	Cluster Tool	Networking	Status
1	Personal Laptop	Minikube	NodePort / minikube tunnel	Completed (Prototype)
2	College Lab Desktops (VMware)	Kubeadm v1.29	Flannel CNI + MetalLB + Istio	Completed
3	HPE ProLiant DL380	Rancher + RKE2	Proxmox NAT + MetalLB + Istio	Active / In Progress

3. Phase 1: Local Prototyping with Minikube

Minikube was used as the entry point into Kubernetes. It simulates a single-node cluster on a local machine using Docker as the backing driver. The purpose of this phase was to understand how the Kubernetes control plane works, how YAML manifests define desired state, and how Pods, Deployments, Services, and the reconciliation loop interact.

Key experiments included deploying a static Nginx welcome page, demonstrating self-healing by deleting a pod and observing the controller manager recreate it, and introducing the Metrics Server to visualise CPU and RAM usage per pod. StatefulSets, DaemonSets, and session affinity were also explored as Kubernetes primitives.

An Istio service mesh was also briefly trialled on top of Minikube to preview canary deployment concepts before they were properly implemented in the kubeadm phase.

This phase is intentionally kept brief in this document. Minikube is a learning environment, not a production architecture. The project's meaningful infrastructure work begins in Phase 2.

4. Phase 2: Multi-Node Cluster on College Lab Desktops (Kubeadm)

- **Environment and Hardware Setup:** Deployed a distributed architecture utilizing two Ubuntu 22.04 virtual machines on a VMware Workstation Type-2 hypervisor using NAT mode to guarantee deterministic IP mapping. The Master Node (`192.168.81.130`) executed the core control plane processes, while the Worker Node (`192.168.81.131`) ran application workloads. Static IP rules were strictly required because regular lab power-downs over weekends would otherwise cause DHCP variations, invalidating the encryption certificates.
- **OS Hardening and Container Runtime:** Prepared both operating systems identically by completely turning off swap memory to prevent initialization failures within the node management software. Internal system routing and kernel flags were modified to enable basic packet forwarding across the hosts so that pods on separate machines could communicate. To serve as the core container engine, a lightweight runtime was installed in place of a full, monolithic Docker installation to start and manage containers cleanly.
- **Cluster Initialization and Node Federation:** Bootstrapped the main control plane directly on the master node, pinning the API server to the host network address and provisioning a dedicated internal range for application traffic. Cross-node communication was activated by deploying the Flannel network plugin, which built a virtual network overlay that wrapped and routed container traffic across the physical machine interfaces. After aligning software versions on the worker

machine, it executed a secure join command using a time-limited token to authenticate and link itself into the active cluster rotation.

5. Phase 2 Networking: MetalLB Bare-Metal Load Balancer

5.1 Why MetalLB Was Needed

In a cloud environment (AWS, GCP, Azure), when a Kubernetes Service is created with `type: LoadBalancer`, the cloud provider automatically provisions an external load balancer and assigns it a public IP address. This is a built-in integration between Kubernetes and the cloud's control plane.

In a bare-metal environment like the college lab, no such integration exists. A `LoadBalancer`-type Service would sit in a permanent `<pending>` state for its external IP, because there is no external system to fulfill the request. This meant Istio's Ingress Gateway — which requires a real external IP to receive traffic — could never start functioning.

MetalLB fills this gap. It is a software-based load balancer designed specifically for bare-metal Kubernetes clusters. It watches for `LoadBalancer`-type Services and assigns them IPs from a pre-configured address pool, then advertises those IPs on the local network so that other machines can reach them.

5.2 Campus Network Constraints

Before configuring MetalLB, the team needed to identify a safe range of IP addresses to use. The college network assigned IPs in the `192.168.81.x` range to lab machines. The Master node was at `.130`, the worker at `.131`.

IP addresses were tested from the Windows host using ping to find addresses in the `.200`–`.210` range that did not respond, confirming they were unallocated. This range was then designated for MetalLB's address pool.

An important network detail: MetalLB in Layer 2 mode works by having one of its speaker pods respond to ARP requests for the assigned VIP. For this to work, `kube-proxy` needed to be configured in `strictARP` mode, which was done by editing the kube-proxy ConfigMap. This ensures that ARP responses for the VIP are only sent by the designated MetalLB speaker and not duplicated by other interfaces.

5.3 IP Address Pool Planning

MetalLB was configured with an `IPAddressPool` resource defining the range `192.168.81.200–192.168.81.210`. An `L2Advertisement` resource was then created to instruct MetalLB to advertise this pool via ARP at Layer 2 of the network stack. This is the simplest MetalLB mode and appropriate for a single Layer-2 network segment like a lab LAN.

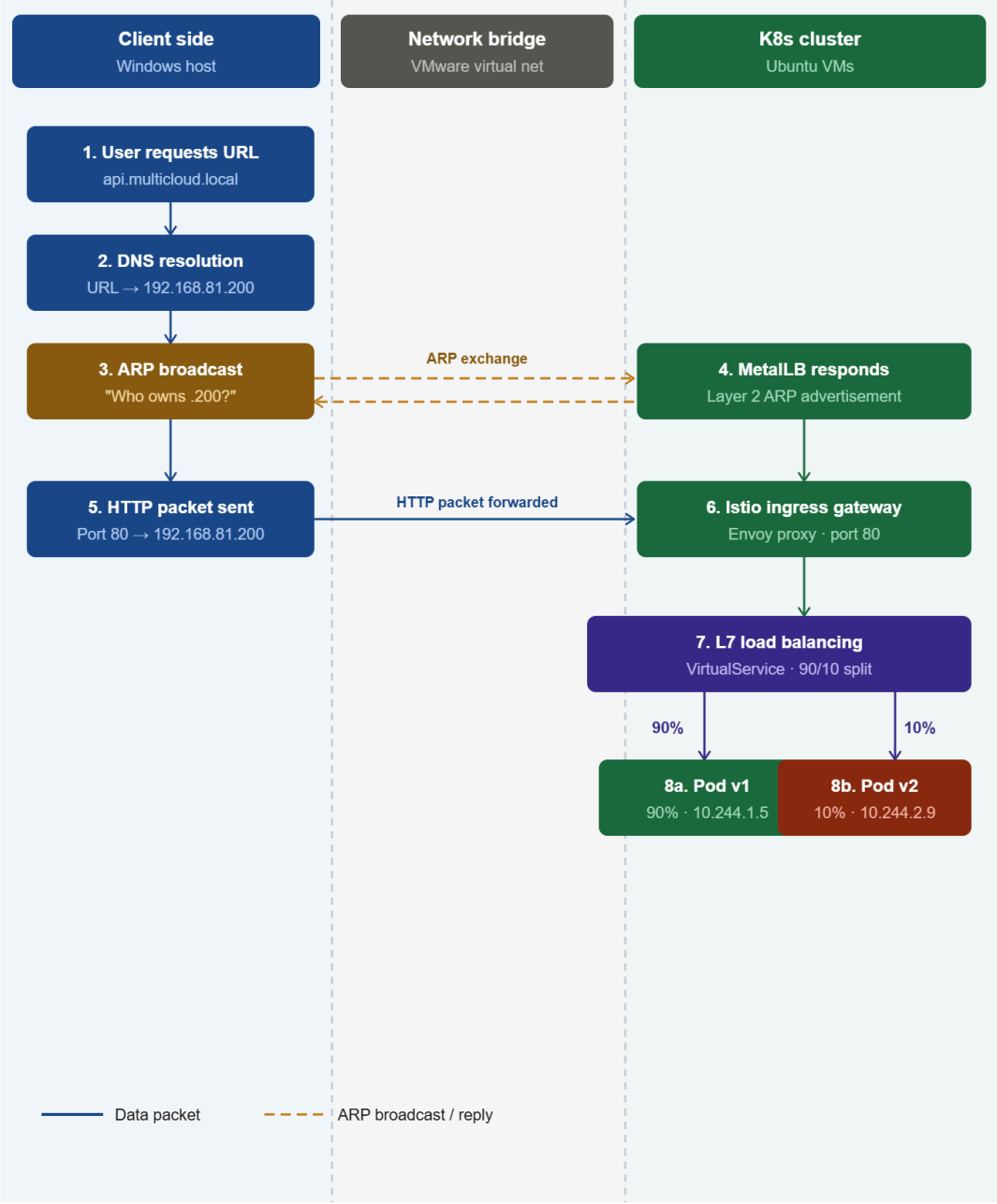
The IP pool size of 10 addresses was sufficient for the project's needs. The Istio Ingress Gateway Service would claim one of these IPs (typically `.200` or `.201`), serving as the single entry point for all external traffic into the cluster.

5.4 MetalLB Architecture and Layer 2 Mode

MetalLB runs as two components inside the cluster:

- **Controller:** A Deployment that watches for `LoadBalancer` type Services and assigns IPs from the pool.
- **Speaker:** A DaemonSet with one pod per node that handles the actual network advertisement. In Layer 2 mode, the speaker on one node (elected as the leader for a given IP) responds to ARP requests from the rest of the network, causing the network switch to forward traffic for the VIP to that node.

The limitation of Layer 2 mode is that all traffic for a given VIP flows through a single node (the ARP responder). There is no true per-packet load balancing at the network level. However, once traffic enters the cluster through that node, Kubernetes' kube-proxy performs in-cluster load balancing across pods.



5.5 Validation and the Bridge Test

To confirm MetalLB was functioning before deploying Istio, a test deployment was created with a LoadBalancer Service. Once MetalLB assigned an external IP (e.g., `192.168.81.201`), the team pinged that address from the Windows host machine. A successful ping confirmed that the "bridge" between the Windows host and the Kubernetes cluster network was alive.

This was a critical checkpoint. The Istio deployment step was not started until this bridge test passed. If the Windows browser cannot reach a MetalLB-assigned IP, the Istio Ingress Gateway is unreachable and the entire traffic routing chain breaks.

---

6. Phase 2 Service Mesh: Istio on Kubeadm

6.1 Why Istio Was Introduced

As the application grew to include multiple microservices (frontend, backend, database, notification service), managing service-to-service communication, security, and traffic routing at the application code level became increasingly complex. Istio addresses this by moving these concerns out of the application code and into the infrastructure layer.

Specifically, Istio was introduced for three reasons:

- **Traffic Management:** The ability to route a defined percentage of traffic to different versions of a service (canary deployment) without changing the application or deploying separate Ingress resources.
- **Security:** Mutual TLS (mTLS) between pods, enforced by the sidecar proxies without any code changes.
- **Observability:** Automatic collection of request metrics, distributed traces, and a visual topology of service traffic using Kiali.

6.2 Istio Installation and the Demo Profile

Istio was installed using its official command-line tool `istioctl`. The `demo` installation profile was chosen. This profile deploys all core Istio components — the control plane (`istiod`), the Ingress Gateway, and the Egress Gateway — with resource limits tuned for non-production environments. It is appropriate for a lab or development cluster where the goal is to have all features available without the overhead of a fully optimized production profile.

The `istiod` component is the Istio control plane. It handles certificate management for mTLS, configuration distribution to all sidecar proxies, and service discovery. When an Istio configuration resource (like a VirtualService) is applied, `istiod` validates and translates it into a format the Envoy proxies can consume, then pushes the updated configuration to all relevant proxies via its xDS API.

6.3 Sidecar Injection and the Envoy Proxy Model

Istio's core mechanism is the sidecar proxy. Every pod in the mesh runs two containers: the application container and an Envoy proxy container injected automatically by Istio. All network traffic in and out of the pod passes through this Envoy proxy. The application container has no awareness of the proxy — it thinks it is communicating directly with other services.

Sidecar injection was enabled for the `default` namespace by labelling it with `istio-injection=enabled`. Once labelled, any new pod created in that namespace automatically has the Envoy sidecar container added to it by a Mutating Webhook Admission Controller before the pod is scheduled.

A pod with a healthy Istio sidecar shows `2/2` under the `READY` column in `kubectl get pods`. If only `1/1` is shown, the sidecar was not injected, which typically means the namespace label was missing or was applied after the pod was already running.

Because Istio's Ingress Gateway is a LoadBalancer-type Service, MetalLB was a prerequisite. Once MetalLB assigned the gateway an external IP, the gateway became the single entry point for all external traffic into the service mesh.

6.4 Istio Gateway — The Cluster Entry Point

The Istio `Gateway` resource defines how traffic enters the mesh at the edge. It configures the Istio Ingress Gateway pod to listen on a specified port (80 for HTTP, 443 for HTTPS) and accept traffic for one or more hostnames.

In the project, a Gateway was configured to accept HTTP traffic on port 80 for the hostname `api.multicloud.local`. The Gateway resource itself does not define where that traffic goes — it is purely an edge listener. Routing decisions are left to the `VirtualService` resource, which references the Gateway.

For the Windows host to resolve `api.multicloud.local` to the MetalLB-assigned IP, an entry was added to the Windows `hosts` file mapping the hostname to the VIP. This simulates DNS resolution without requiring an actual DNS server.

6.5 Traffic Management — DestinationRule and VirtualService

Istio's traffic management is implemented through two Custom Resource Definitions:

- **DestinationRule** defines how traffic is handled after it is routed to a service. Most importantly for this project, it defines *subsets* — named groups of pods differentiated by their Kubernetes labels. For the canary deployment, two subsets were defined: `v1` (pods labelled `version: v1`) and `v2` (pods labelled `version: v2`). Without a DestinationRule, Istio cannot distinguish between v1 and v2 pods of the same application.
- **VirtualService** defines the routing logic. It attaches to a Gateway, listens for requests to a specific hostname, and then distributes traffic across the subsets defined in the DestinationRule according to percentage weights. The VirtualService is the component where the 90/10 canary split is actually specified.

The relationship between these components forms a chain:

Gateway (accepts traffic at the edge) → VirtualService (applies routing rules) → DestinationRule (identifies the backend pod groups) → Kubernetes Service (delivers to an individual pod).

6.6 Canary Deployment

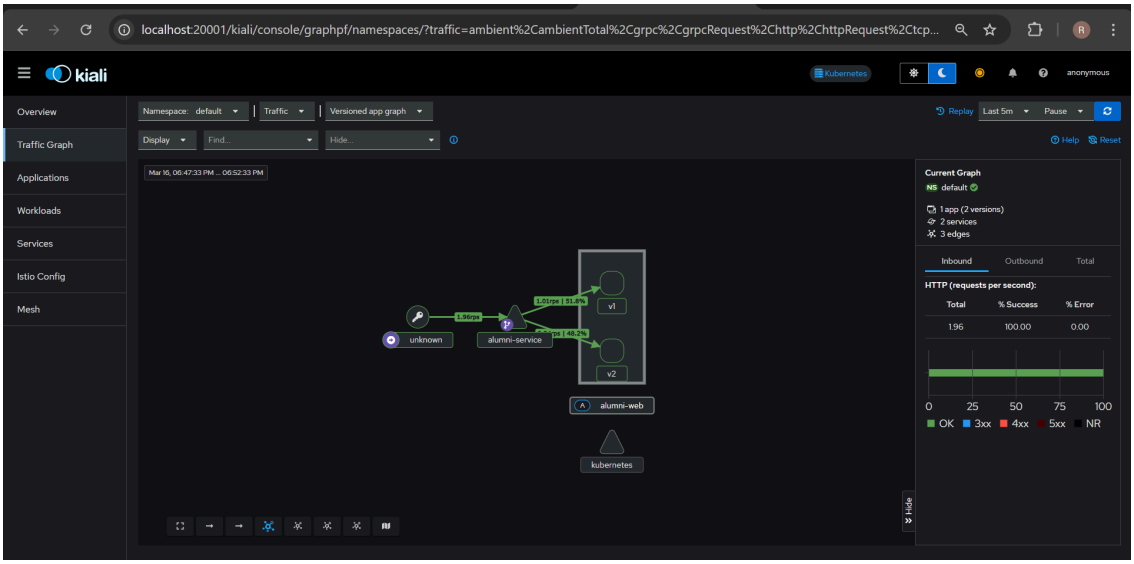
A canary deployment is a release strategy where a new version of an application is initially exposed to only a small fraction of users. If no errors are observed, the traffic weight is gradually shifted until the new version carries 100% of traffic and the old version is decommissioned.

In this project, two versions of the FastAPI-based backend were deployed simultaneously — `v1` (the stable version) and `v2` (the canary version). The VirtualService was configured to send 90% of all incoming requests to `v1` and 10% to `v2`.

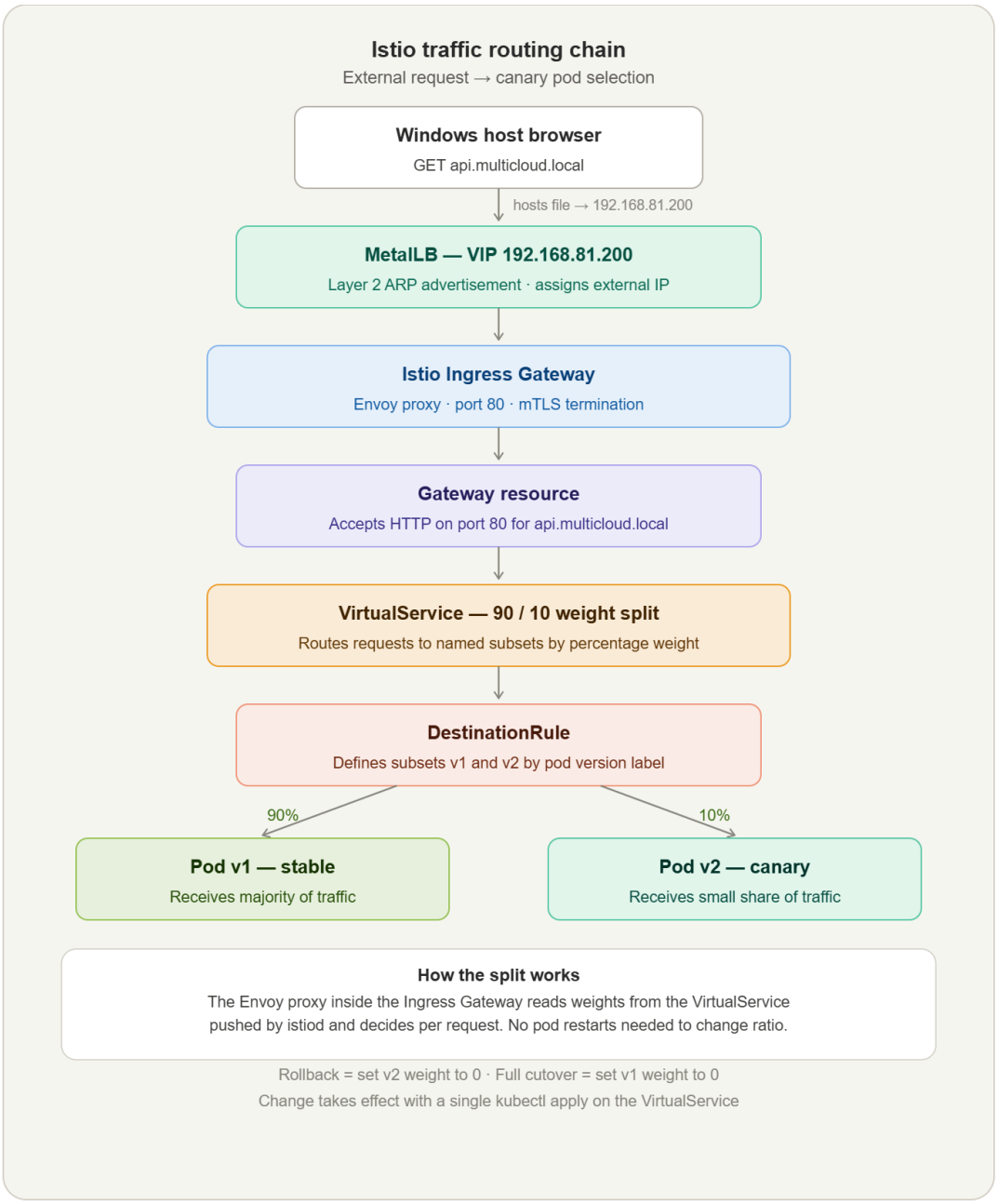
This traffic split operates at the proxy level — the Envoy sidecar in the Ingress Gateway pod makes the routing decision per request, consulting the weights in the VirtualService configuration. It does not require different numbers of replicas for each version. Even with one replica each, the 90/10 distribution is maintained statistically over many requests.

To validate the split, a PowerShell loop was run on the Windows host, sending continuous HTTP requests to the application and printing the response. Over many iterations, approximately 9 in 10 responses came from v1 and 1 in 10 from v2, confirming the canary logic was working correctly.

The traffic weight in the VirtualService is the only configuration that needs to change to progress the canary. Moving from 90/10 to 50/50 to 0/100 can be done with a single `kubectl apply` on an updated VirtualService, with no pod restarts or infrastructure changes required. Similarly, a rollback is just as simple — setting the v2 weight back to 0 immediately stops all traffic to the problematic version.



6.7 Istio Traffic Routing Chain



- **Edge Client Request:** An external user initiates a connection from a Windows host browser targeting the application domain `api.multicloud.local`.
- **Local Name Resolution:** The client computer bypasses public DNS servers by consulting a static mapping entry in its local `hosts` file, resolving the hostname straight to a pre-allocated MetalLB Virtual IP address (`192.168.81.200`).
- **Bare-Metal Load Balancing:** MetalLB intercepts the network-level request. Operating via Layer 2 ARP advertisements, it dynamically maps the requested VIP to a specific physical node within the cluster.

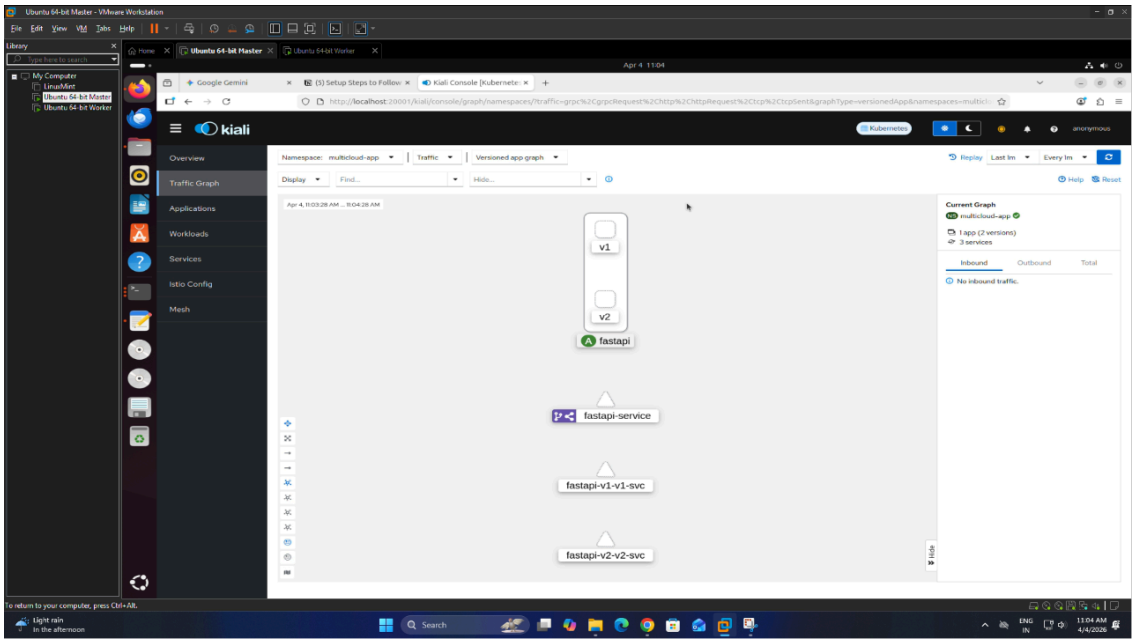
- **Service Mesh Edge Ingress:** The packet hits the `Istio Ingress Gateway` pod. An underlying Envoy proxy process intercepts the incoming traffic stream on port `80` and manages mutual TLS (mTLS) decryption and session termination.
- **Application Layer Decoupling:** Traffic handling transitions from raw network objects into declarative Istio Custom Resource Definitions (CRDs):
  - **Gateway Resource:** Acts as the cluster's edge listener, validating the incoming HTTP port criteria and filtering against the allowed `api.multicloud.local` host string.
  - **VirtualService Controller:** Functions as the primary logical traffic router. It reads configuration rules pushed by the control plane (`istiod`) and applies a statistical weight distribution, splitting the request pool.
  - **DestinationRule Subsets:** Evaluates the target pods based on internal version metadata labels, grouping them into explicit runtime groups (`v1` and `v2`).
- **Canary Workload Balancing:** The Ingress Gateway's Envoy proxy distributes requests dynamically at a per-request level without requiring any deployment adjustments or pod scaling. It steers a precise **90%** slice of overall traffic down to the stable `Pod v1` pool and a restricted **10%** slice to the testing canary `Pod v2` pool. Adjusting or reverting this ratio is handled through a single `kubectl apply` action on the VirtualService manifest.

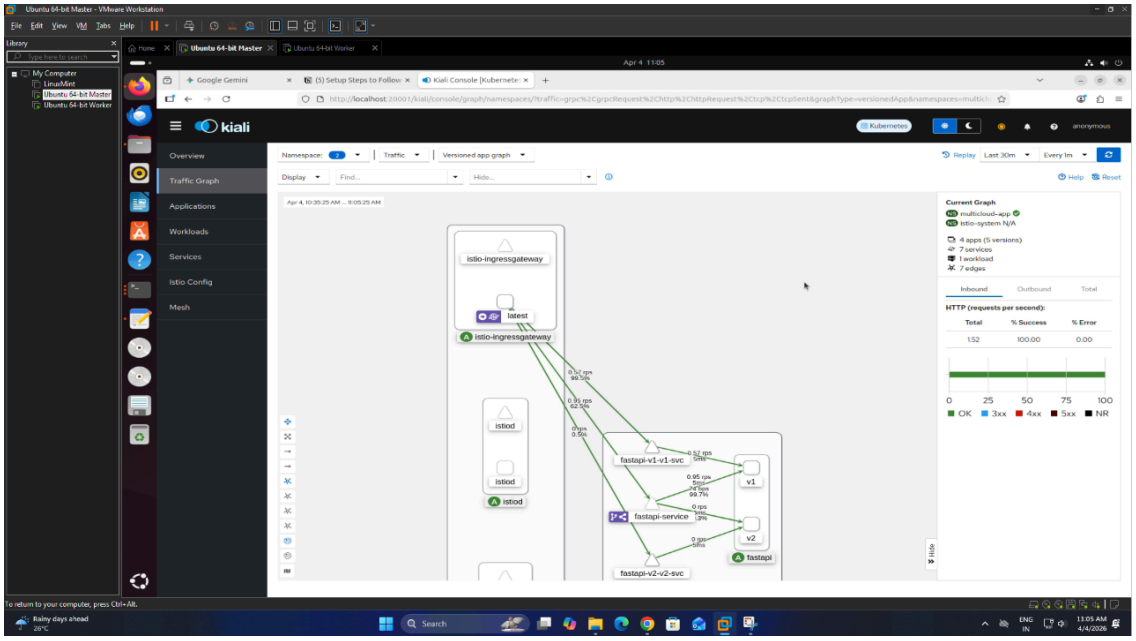
6.8 Observability: Kiali, Prometheus, and the Mesh Dashboard

Kiali is Istio's dedicated topology and configuration dashboard. It consumes metrics scraped by Prometheus from the Envoy sidecar proxies and renders a real-time graph of service-to-service traffic within the mesh.

In the Kiali graph for this project, the traffic flow is visually represented: external traffic enters through the Istio Ingress Gateway, flows into the alumni service, and then splits into two paths, one to `v1` pods and one to `v2` pods, with the percentage weights displayed on each edge. This provides instant visual confirmation that the canary split is behaving as configured.

Prometheus was deployed from Istio's official addon manifests. It automatically discovers the Envoy proxies across all pods in the mesh and scrapes their metrics endpoints. These metrics include request rates, error rates, latency percentiles, and traffic volumes; all the data needed for the Grafana dashboards and alert rules.





7. Application Packaging: Helm Charts and Sonatype Nexus

7.1 Helm as a Deployment Standard

As the number of Kubernetes resources grew (Deployments, Services, ConfigMaps, Istio VirtualServices, DestinationRules, Gateways), managing them as individual YAML files became impractical. Helm was introduced as the package manager for Kubernetes resources.

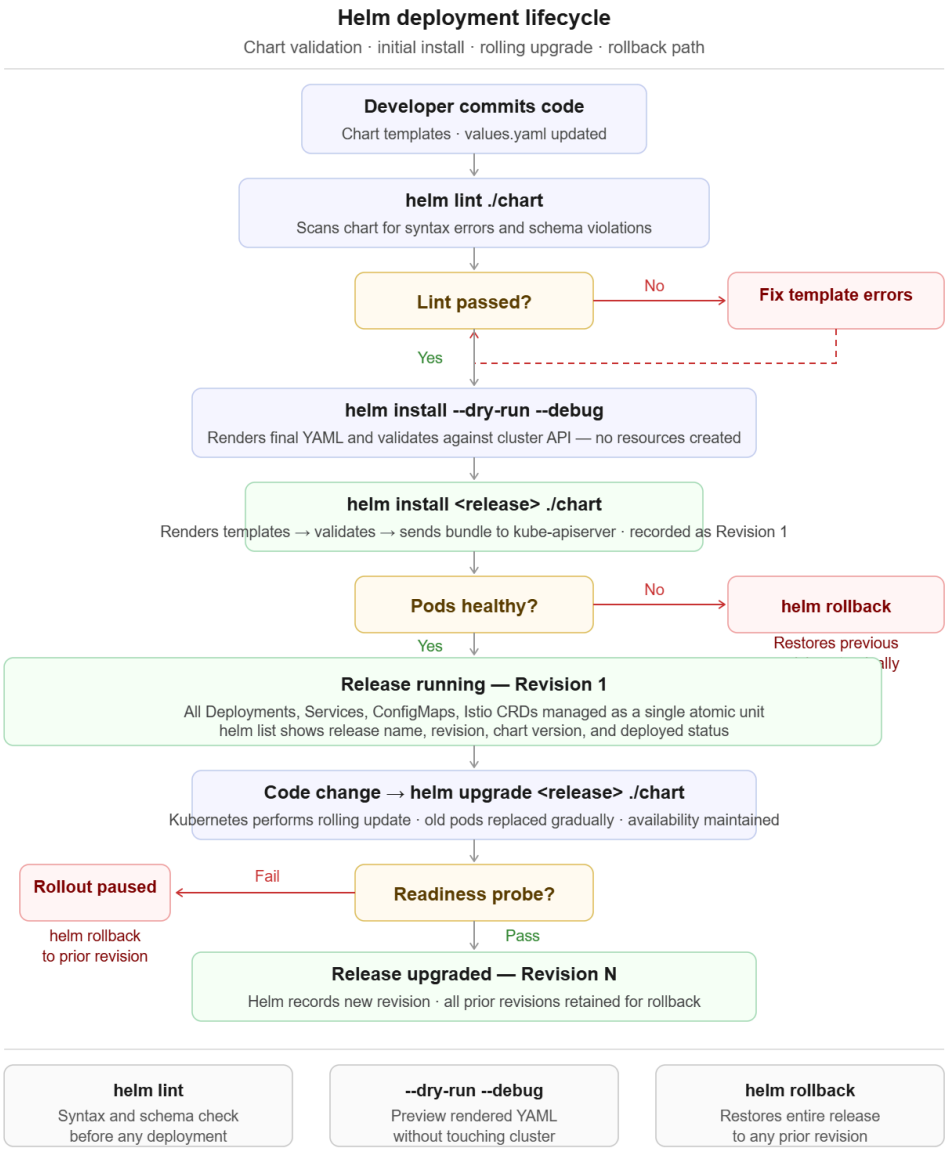
A Helm chart is a directory of template files and a values file. The templates define Kubernetes resources using Go templating syntax, where variables are replaced at deploy time with values from

`values.yaml`. This means that deploying v1 and v2 of the application does not require two separate sets of nearly identical YAML files — the same chart is installed twice with different `version` parameters overriding the relevant template variables.

The project chart structure followed the standard Helm layout: `Chart.yaml` for metadata, `values.yaml` for default configuration variables (image repository, tag, port, version, replica count), and a `templates/` directory containing the Deployment and Service definitions. Istio rules were managed in a separate directory to keep application and traffic logic separate.

Helm also provides lifecycle management. A Helm release tracks the entire set of resources belonging to a deployment as a single unit. If a deployment fails mid-way, Helm can report the failure clearly. An upgrade that introduces a bug can be instantly reverted with `helm rollback`, restoring the entire release to its previous state without manually identifying which resources changed.

Before deploying, `helm lint` was used to validate the chart for syntax errors, and `helm install --dry-run --debug` was used to preview the rendered YAML without touching the cluster. This allowed the team to catch template errors before any resources were created.



7.2 Sonatype Nexus as the Private Registry

Rather than relying on public container registries like Docker Hub, Sonatype Nexus Repository Manager was deployed as the team's private registry. Nexus stores the Docker images for the alumni application's frontend and backend, ensuring that the cluster pulls images from a controlled, internal source.

Nexus was run as a Docker container on the host machine, exposing both its web UI (port 8081) and its Docker registry connector (port 5000). The registry was initialized with HTTP rather than HTTPS for simplicity in a lab environment. This required an explicit insecure registry configuration when starting Minikube and a similar configuration in containerd on the kubeadm nodes.

The Kubernetes cluster itself needed credentials to pull images from Nexus. This was achieved by creating a Kubernetes Docker registry Secret. When a Pod's spec references this Secret as an

`imagePullSecret`, the kubelet uses those credentials to authenticate with Nexus before pulling the image. Without this, the Pod would fail with an `ImagePullBackOff` error.

8. Application Hosting and Internal Service Communication

8.1 Deploying LocalStack on the Multi-Node Cluster

After successfully establishing the kubeadm-based multi-node Kubernetes cluster, the next objective was to deploy a real cloud-native workload for infrastructure validation.

LocalStack was introduced to emulate AWS cloud services locally within the college lab environment. This allowed the infrastructure to provide AWS-compatible services without depending on external cloud providers.

The deployment included support for services such as:

- Amazon S3
- IAM
- DynamoDB
- KMS

Running LocalStack inside the distributed Kubernetes cluster helped validate:

- Pod scheduling across nodes
- Internal Kubernetes networking
- Cross-node communication
- Service discovery using Kubernetes DNS

This marked the transition from basic cluster setup to functional application hosting.

8.2 FastAPI Backend Integration

To interact with LocalStack services programmatically, a FastAPI backend application was deployed inside the Kubernetes cluster.

The FastAPI application exposed REST APIs that allowed interaction with LocalStack services through standard HTTP requests. The backend was designed to simulate a lightweight cloud management API layer operating inside the Kubernetes environment.

The application provided endpoints for operations such as:

- Creating S3 buckets
- Listing existing buckets
- Uploading objects into buckets
- Retrieving bucket contents
- Deleting buckets and objects

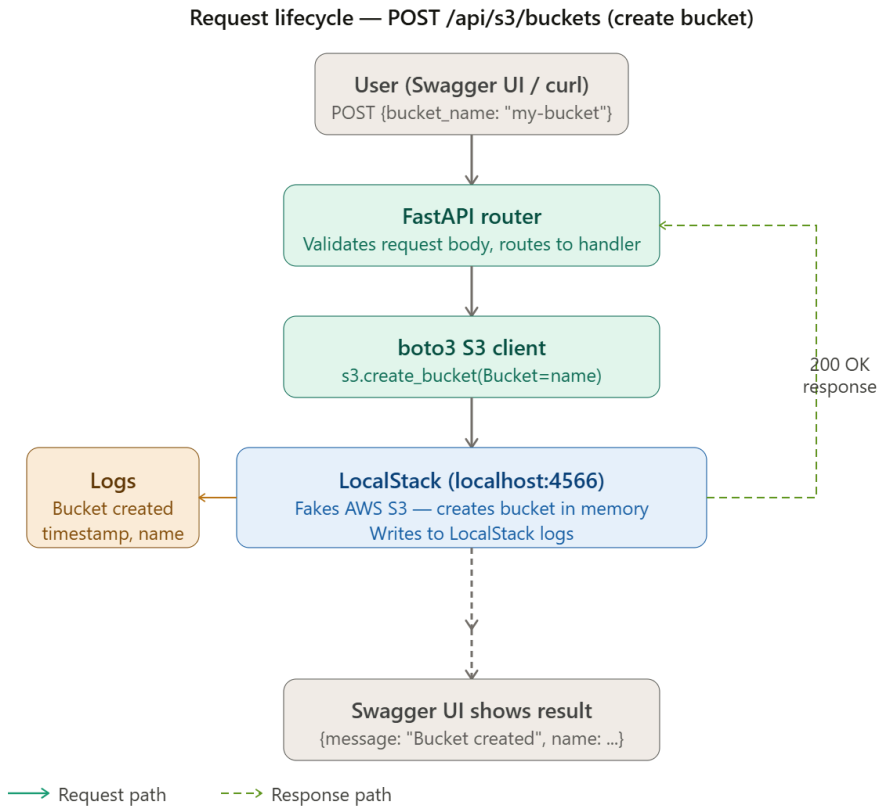
Communication between the FastAPI backend and LocalStack services occurred internally through Kubernetes Service networking and DNS-based service discovery. This validated that workloads running on different nodes inside the cluster could reliably communicate with one another.

The FastAPI application also automatically exposed Swagger/OpenAPI documentation through the `/docs` endpoint. This provided a browser-based interface for testing APIs directly from the Windows host machine and simplified validation of backend functionality.

The successful operation of the FastAPI backend demonstrated that:

- Kubernetes Services were functioning correctly
- Internal cluster networking was stable
- Cross-node service communication was operational
- Application pods could successfully interact with LocalStack services
- The multi-node infrastructure was capable of hosting real application workloads

This deployment served as one of the first complete application-layer validations of the kubeadm-based infrastructure environment.



9. Phase 3: Migration to HPE ProLiant ML 110 (Proxmox + Rancher)

9.1 Hardware: HPE ProLiant ML 110

The project's primary target hardware is an HPE ProLiant ML 110 Gen 11 server. Unlike the shared lab desktops, this server is dedicated to the project, runs continuously, and provides the stable compute foundation needed for performance benchmarking.

The server has two network interfaces of interest:

- **nic0 (192.168.41.38):** The primary NIC connected to the college LAN. This is the only MAC address whitelisted by the campus firewall and the path through which all outbound internet traffic flows.
- **iLO NIC (192.168.41.63):** HPE's Integrated Lights-Out management NIC, used for remote server management (BIOS, power, console) and not for workload traffic.

9.2 Bare-Metal Hypervisor: Proxmox VE

Proxmox Virtual Environment (Proxmox VE) was installed directly on the ProLiant's bare metal as the Type-1 hypervisor. Unlike VMware Workstation (which requires a host OS), Proxmox runs on the hardware directly using the Linux KVM subsystem and QEMU for virtualization.

The choice of Proxmox over alternatives like VMware ESXi was driven by several factors: it is open-source, has a web-based management interface accessible on port 8006 from the college LAN, supports VirtIO paravirtualized drivers for near-native network and disk performance, and allows hardware CPU features to be passed directly through to VMs.

VirtIO networking was specifically important for the project's benchmarking goals. Because VirtIO bypasses the emulation overhead of traditional virtual NICs, the network performance metrics collected from the cluster reflect the capabilities of the physical hardware more accurately, reducing noise in the evaluation data.

9.3 Virtual Machine Layout

Three VMs were provisioned on Proxmox, all connected to an internal virtual bridge (`vmbr1`) on the `10.10.10.0/24` subnet:

VM	Role	IP Address
vm-rancher	Rancher Management Server	10.10.10.50
k8s-master	Kubernetes Master Node	10.10.10.51
k8s-worker	Kubernetes Worker Node	10.10.10.52

All VMs run Ubuntu Server (headless, no desktop environment), which reduces resource overhead and is standard practice for server workloads. They were configured with VirtIO disk controllers and network adapters to maximize I/O throughput.

9.4 Campus Network Challenge: MAC Address Filtering

This was the most significant networking challenge of Phase 3 and required a custom infrastructure solution.

The college campus network enforces strict MAC address filtering at the Layer-2 switch level. Only devices with pre-registered MAC addresses are permitted to send traffic. The ProLiant's `nic0` MAC

address (`7c:a6:2a:...`) is whitelisted, meaning it is the only MAC address the campus switch will forward traffic from.

The VMs running on Proxmox each have their own virtual MAC addresses generated by KVM. If a VM's traffic reached the campus switch with its virtual MAC as the source address, the packet would be silently dropped. This means none of the three VMs (k8s-master, k8s-worker, vm-rancher) can communicate with the internet or even the college LAN directly.

This constraint fundamentally required a network architecture change. The VMs could not be bridged directly to the physical LAN.

9.5 Custom NAT Gateway with vmbr1

To overcome the MAC filtering constraint, a custom two-bridge NAT architecture was designed using Proxmox's Linux bridge capabilities and the Linux kernel's `iptables`.

vmbr0 is the bridge connected to the physical `nic0`. Only Proxmox itself uses this bridge for external communication. Traffic leaving through `vmbr0` carries `nic0`'s whitelisted MAC address and is allowed through the campus switch.

vmbr1 is an entirely internal virtual bridge. It exists only inside the Proxmox host and is not connected to any physical interface. All three VMs are connected to `vmbr1` and receive IPs in the `10.10.10.0/24` range. This private subnet is invisible to the campus network.

iptables MASQUERADE is configured on the Proxmox host's kernel. When a VM (e.g., `k8s-worker` at `10.10.10.52`) sends a packet destined for an external address, the packet first goes through `vmbr1` to the Proxmox host's kernel. The `iptables MASQUERADE` rule performs Source NAT (SNAT), rewriting the packet's source IP and MAC to that of `nic0` (`192.168.41.38`). The packet then exits through `vmbr0` to the campus switch, which sees only the whitelisted `nic0` MAC and allows it through.

Return traffic from the internet arrives back at `nic0`, and the NAT table in the kernel maps the connection back to the correct VM and delivers the response through `vmbr1`.

This architecture provides two important properties:

- **Security bypass:** All VM traffic appears to originate from a single, whitelisted host IP. The campus network is unaware of the VMs' existence.
- **Network isolation:** The Kubernetes nodes are on a private, isolated subnet. They cannot be directly accessed from the college LAN, which reduces the attack surface.

Inbound access (e.g., from a lab PC browser to the cluster) is handled by DNAT rules, which forward traffic arriving at a specific port on `nic0` to the appropriate VM.

---

10. Phase 3 Cluster Management: Rancher and RKE2

10.1 Why Rancher Replaced Kubeadm

The kubeadm-based cluster from Phase 2 had several operational limitations. Cluster lifecycle operations (upgrades, node additions, certificate renewals) were entirely manual. There was no centralized UI to inspect cluster health, manage workloads across namespaces, or configure RBAC policies without using the command line directly. Debugging required SSH access to each node.

Rancher by SUSE is a production-grade Kubernetes cluster manager that addresses these limitations. It provides a web-based control plane for managing one or more Kubernetes clusters, with built-in support for cluster provisioning, lifecycle management, monitoring integration, and access control. In the context of this project, transitioning to Rancher reflects the move from a prototype setup to an architecture representative of how Kubernetes is actually managed in enterprise environments.

10.2 Rancher Architecture on Proxmox

Rancher runs on its own dedicated VM (`vm-rancher` at `10.10.10.50`) within the Proxmox cluster. It is accessed via its web interface at port 8006 (or its own configured NodePort). Rancher maintains a management cluster internally using RKE2 or K3s, and registers the downstream `k8s-master` / `k8s-worker` cluster for management.

Rancher communicates with the downstream cluster's API server via the internal `10.10.10.x` network. From the Rancher UI, operators can view node health, inspect pod logs, manage namespaces, configure storage, and deploy Helm chart applications from a catalogue — all without requiring direct `kubectl` access.

10.3 RKE2: The Production-Grade Kubernetes Distribution

The downstream cluster on the ProLiant is provisioned using RKE2 (Rancher Kubernetes Engine 2), which is Rancher's own CNCF-compliant Kubernetes distribution. RKE2 differs from kubeadm-bootstrapped clusters in several ways:

- It bundles all control plane components as a single binary, simplifying installation and upgrades.
- It is FIPS 140-2 compliant by default and uses containerd as the container runtime with hardened defaults.
- Cluster certificates are managed automatically, including rotation, removing a significant operational burden.

When a node is added to a Rancher-managed RKE2 cluster, Rancher generates a node registration command that, when run on the target VM, installs the RKE2 agent and establishes a secure connection back to the Rancher management server. No manual certificate distribution or token management is required.

---

11. Security Integration

11.1 Keycloak as the Identity Provider

Keycloak was deployed as the centralized Identity Provider (IdP) for the project. It manages user registration, authentication, and role assignment through the OpenID Connect (OIDC) protocol. Keycloak's concept of a "realm" provides an isolated namespace for the project's users, clients, and roles.

Two integration paths were implemented: one for administrative tooling (Jenkins and Nexus) and one for the application's end users.

11.2 OAuth2-Proxy and Nexus OIDC Gate

Sonatype Nexus(OSS) does not natively support OIDC-based login. To enforce centralized authentication for Nexus, an `OAuth2-Proxy` was deployed as a Kubernetes-native gateway in front of Nexus, listening on port 30004.

When a user attempts to access Nexus, the proxy intercepts the request and redirects them to Keycloak for login. Keycloak verifies the credentials and returns a JWT (JSON Web Token). The OAuth2-Proxy validates the JWT and forwards the user's identity to Nexus via a trusted request header. Nexus's "Rut Auth" (Remote User Token) realm was configured to trust this header and treat the forwarded username as an authenticated session, bypassing Nexus's internal login screen.

Keycloak roles were mapped to Nexus external roles via RBAC synchronization, so a user with the appropriate Keycloak role automatically receives `nx-admin` privileges in Nexus without any manual configuration in Nexus itself.

Jenkins authentication was handled via JSON Service Account tokens, enabling secure non-interactive API access for the CI/CD pipeline without embedding plain-text credentials in pipeline scripts.

11.3 Application-Level RBAC

Within the alumni application itself, role-based access control was implemented using Keycloak's role mapping and JWT Bearer Token validation at the API gateway level.

Five roles were defined: `student`, `alumni`, `mentor`, `recruiter`, and `admin`. Each role has a scoped set of permissions. A new user who registers enters a `pending` state and cannot access protected resources until an admin approves their account. Keycloak issues JWTs containing the user's role, and the Node.js backend validates these tokens on every protected API request. The Keycloak Subject ID (a stable UUID) was used as the persistent user identifier across all services, ensuring reliable identity correlation regardless of username or email changes.

---

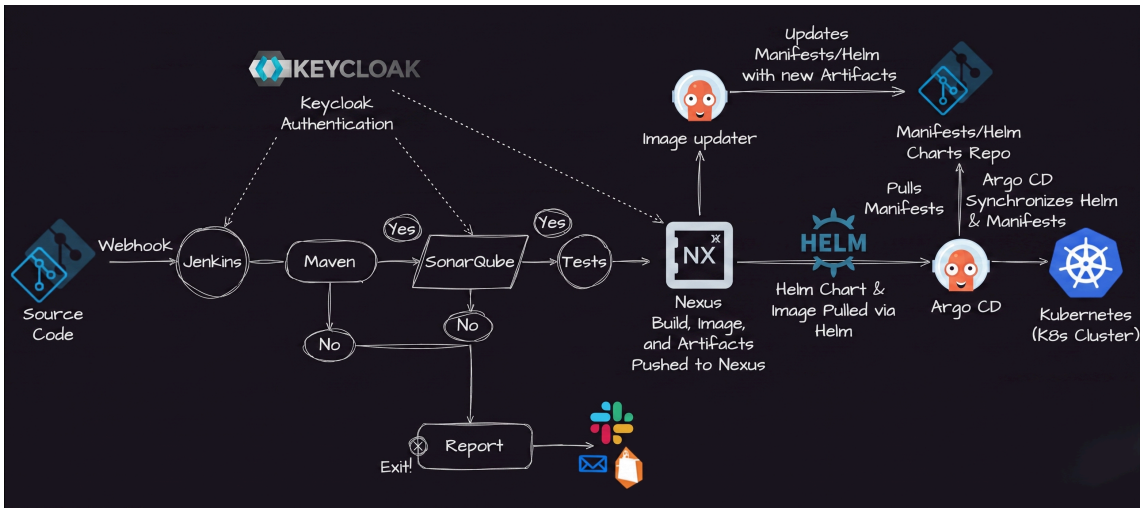
12. CI/CD Pipeline

The project's continuous integration and deployment pipeline is built around Jenkins as the automation server, with GitHub Actions providing lightweight pre-merge validation.

When a developer opens a pull request, GitHub Actions runs ESLint to check for syntax and style violations before the code can be merged. This prevents obviously broken code from entering the main branch.

After merging, Jenkins takes over. The pipeline pulls the updated code, executes Vitest unit tests for the Node.js microservices, runs SonarQube static analysis to enforce code quality gates (blocking the build if vulnerabilities or excessive technical debt are detected), builds Docker images for each changed service, and pushes those images to the Nexus private registry.

Deployment is performed via `helm upgrade` on the appropriate Helm chart. This triggers Kubernetes to perform a rolling update — gradually replacing old pods with new ones while maintaining availability. If the new pods fail their readiness probes, Kubernetes automatically pauses the rollout, and Helm can be used to roll back to the previous revision.



13. Observability and Alerting Stack

The observability stack consists of four components deployed via the Prometheus Operator:

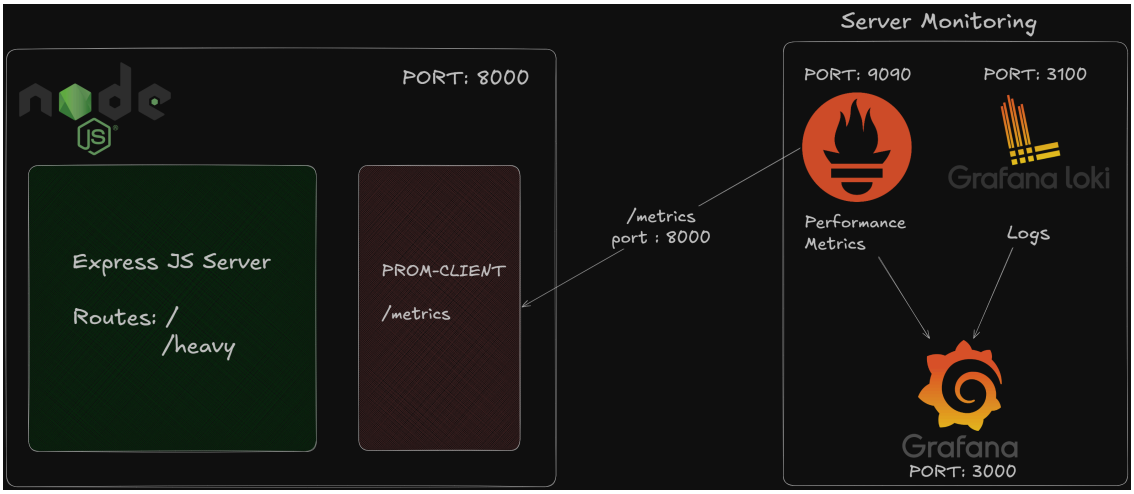
Prometheus scrapes metrics from all Kubernetes nodes, pods, and the Envoy sidecar proxies at regular intervals. The Prometheus Operator uses Custom Resource Definitions (CRDs) to automatically discover new scrape targets as pods are added or removed, eliminating the need to manually update the Prometheus configuration.

Loki is the log aggregation backend. It collects container logs from all pods and indexes them for time-range queries. Unlike Prometheus which stores numeric time-series data, Loki stores and indexes log text, making it possible to search for specific error messages or stack traces across all containers.

Grafana provides the unified visualization layer. It connects to both Prometheus (for metrics) and Loki (for logs) and renders them in shared dashboards. Key dashboards show node CPU and memory utilization, pod-level resource consumption, request rates and error rates per service, and Istio mesh traffic flow.

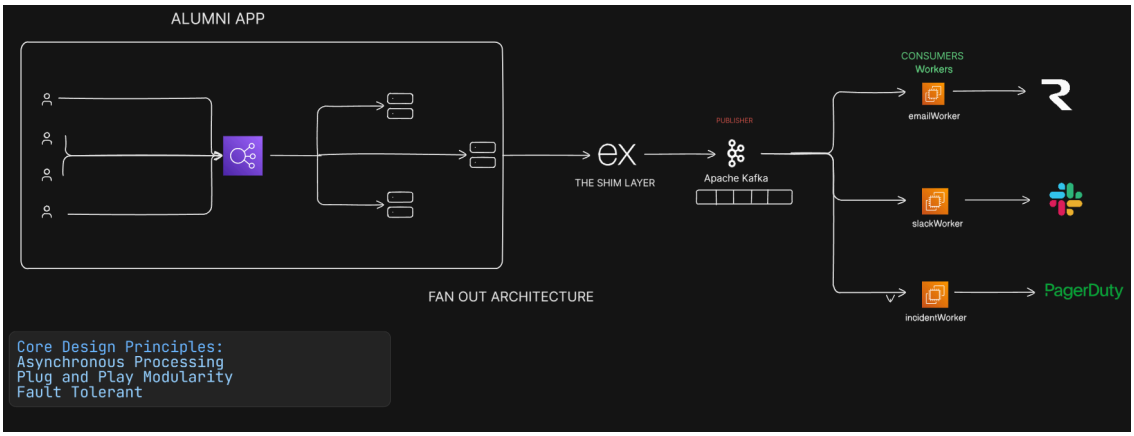
Alertmanager receives alerts from Prometheus based on defined rules (e.g., CPU usage exceeds 85% for 5 minutes, a pod has restarted more than 3 times, an API's 99th-percentile latency exceeds a threshold). Alertmanager groups, deduplicates, and routes these alerts via a Webhook to the project's custom Notification Microservice.

Monitoring SLA Breaches: The observability stack continuously tracks system health and application metrics. Alertmanager is configured to evaluate these metrics against strict Service Level Agreements (SLAs). Alerts are instantly triggered for critical SLA breaches, such as unexpected API latency spikes, high error rates, dropped packets, or pod crash loops. Once Alertmanager detects an SLA breach, it routes the event via a webhook to a custom, stateless Node.js Notification Microservice.

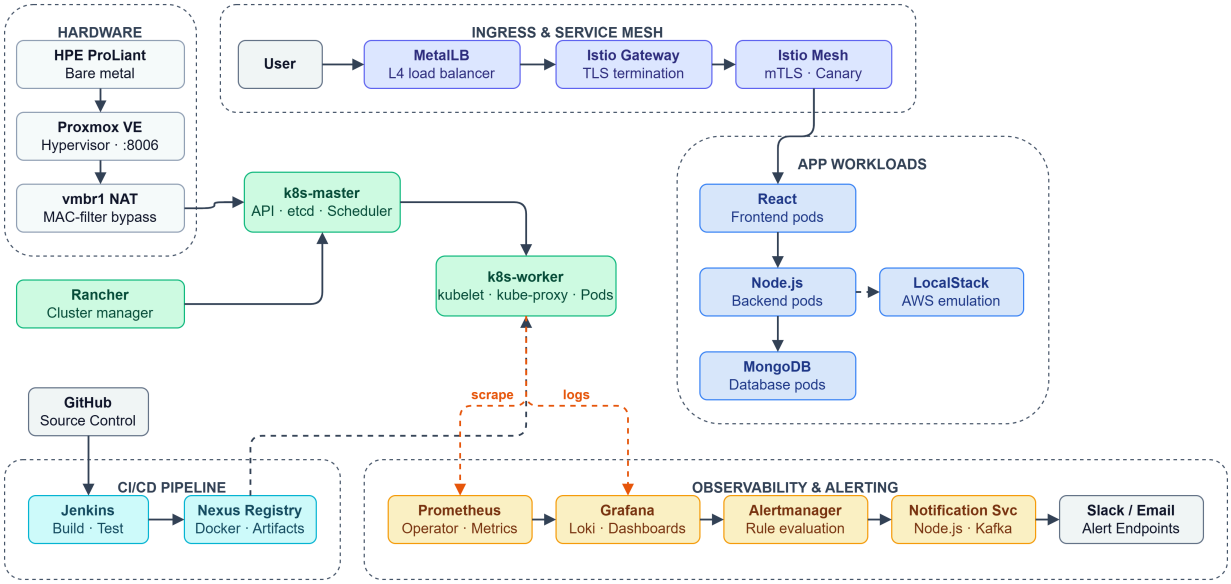


14. Notification Microservice

The Notification Microservice is a stateless Node.js service that uses Apache Kafka as a message broker. When an alert webhook arrives, the service publishes it to a Kafka topic. Consumers subscribed to that topic fan out the alert to designated Slack channels and email endpoints. This event-driven architecture decouples the alerting source from the notification destinations and ensures alerts are not lost even if one notification channel is temporarily unavailable.



15. High-Level System Architecture

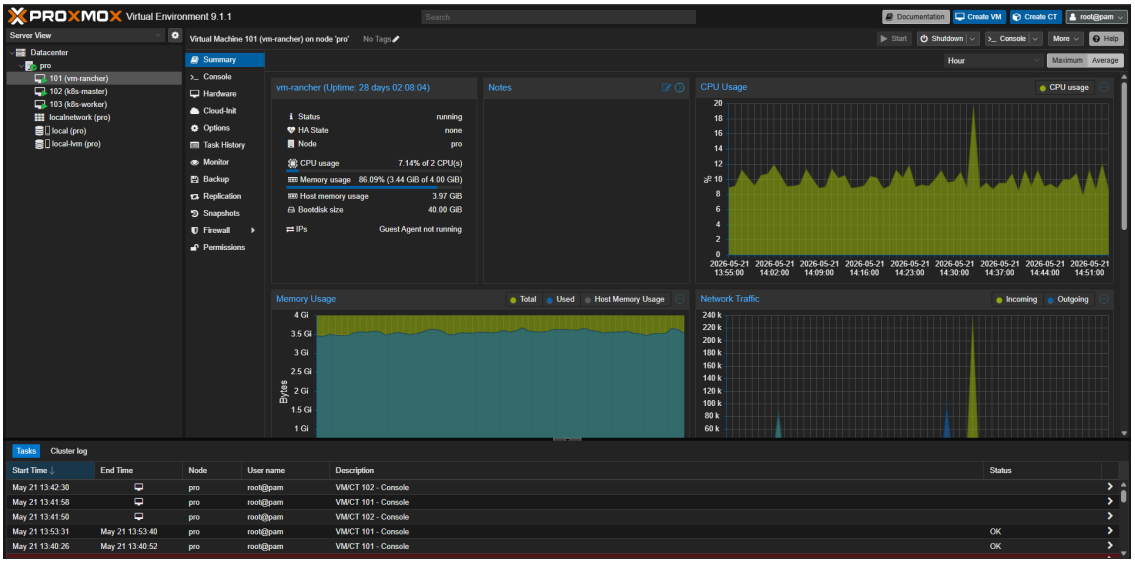


The following diagram summarizes the full project architecture at its current state:

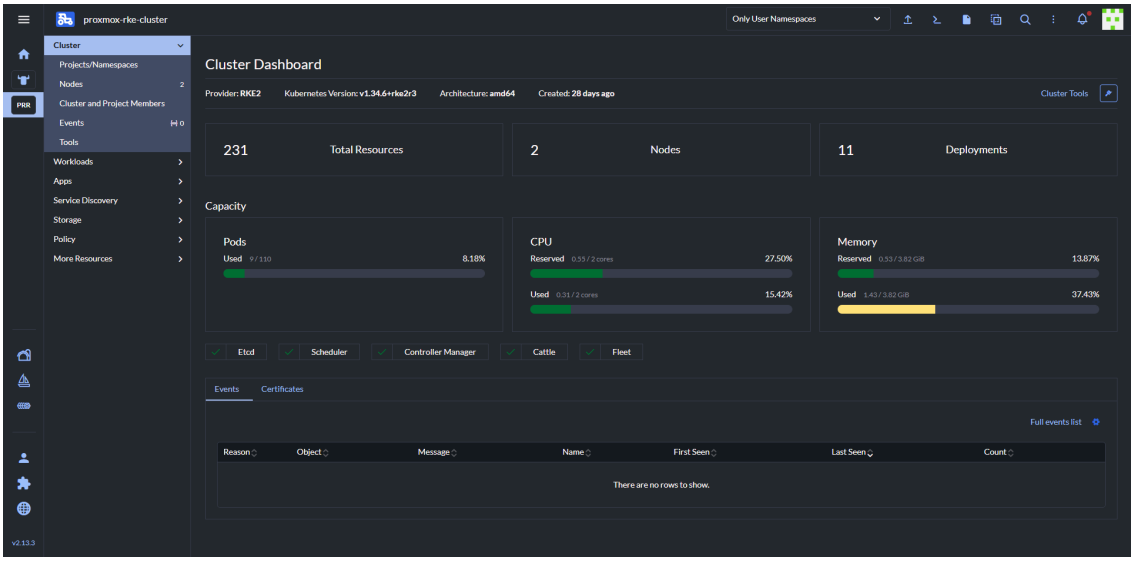
- **Hardware Layer:** HPE ProLiant ML 110 bare metal running Proxmox VE
- **Virtualization Layer:** Three KVM VMs (vm-rancher, k8s-master, k8s-worker) on the `10.10.10.0/24` vmbr1 subnet
- **Network Layer:** iptables NAT on Proxmox host routing VM traffic through the whitelisted nic0 MAC address; MetalLB providing external IPs in the `192.168.81.200–210` range
- **Ingress Layer:** MetalLB → Istio Ingress Gateway → Istio Mesh (mTLS + Canary routing)
- **Application Layer:** React frontend, Node.js backend, MongoDB, LocalStack (AWS emulation)
- **CI/CD Layer:** GitHub → Jenkins → Nexus Registry → Helm Upgrade
- **Observability Layer:** Prometheus → Grafana (with Loki + Alertmanager) → Notification Microservice → Slack/Email.

16. Screenshots

Proxmox UI:


PROXMOX Virtual Environment 9.1.1
Server View
Datacenter
pro
101 (vm-rancher)
102 (k8s-master)
103 (k8s-worker)
localnetwork (pro)
local (pro)
local-lvm (pro)
Virtual Machine 101 (vm-rancher) on node 'pro'
Summary
Console
Hardware
Cloud-Init
Options
Task History
Monitor
Backup
Replication
Snapshots
Firewall
Permissions
vm-rancher (Uptime: 28 days 02:08:04)
Status running
HA State none
Node pro
CPU usage 7.14% of 2 CPU(s)
Memory usage 86.09% (3.44 GiB of 4.00 GiB)
Host memory usage 3.97 GiB
Bootdisk size 40.00 GiB
IPs Guest Agent not running
Notes
CPU Usage
Memory Usage
Network Traffic
Tasks
Cluster log
Start Time
End Time
Node
User name
Description
Status
May 21 13:42:30 pro root@pam VM/CT 102 - Console
May 21 13:41:58 pro root@pam VM/CT 101 - Console
May 21 13:41:50 pro root@pam VM/CT 102 - Console
May 21 13:53:31 May 21 13:53:40 pro root@pam VM/CT 101 - Console OK
May 21 13:40:26 May 21 13:40:52 pro root@pam VM/CT 101 - Console OK

Rancher UI:


proxmox-rke-cluster
Only User Namespaces
Cluster
Projects/Namespaces
Nodes
Cluster and Project Members
Events
Tools
Workloads
Apps
Service Discovery
Storage
Policy
More Resources
Cluster Dashboard
Provider: RKE2
Kubernetes Version: v1.34.6+rke2r3
Architecture: amd64
Created: 28 days ago
Cluster Tools
231 Total Resources
2 Nodes
11 Deployments
Capacity
Pods
Used 9 / 110 8.18%
CPU
Reserved 0.55 / 2 cores 27.50%
Used 0.31 / 2 cores 15.42%
Memory
Reserved 0.53 / 3.82 GiB 13.87%
Used 1.43 / 3.82 GiB 37.43%
Etcd
Scheduler
Controller Manager
Cattle
Fleet
Events
Certificates
Full events list
Reason
Object
Message
Name
First Seen
Last Seen
Count
There are no rows to show.
v2.13.3